

Section 3: Application — Answer Key. Answers shown in amber boxes. Q18 is a stretch question; accept any response that captures the core idea of cooperative yielding at await points.

Section 3: Application

Questions 15 – 18

15. Write a complete program that turns all NeoPixels green when the board is tilted UP, and turns them off otherwise. Include all four patterns from the Primer.

ANSWER

A complete, working program using all four patterns — import, create, loop, decide.

```
import pykit_explorer
from imu_sensor import IMUSensor
from neopixels import NeoPixels, Colors

imu = IMUSensor()
px = NeoPixels()

while True:
    direction = imu.tilt_direction()
    if direction == "UP":
        px.fill(Colors.GREEN)
    else:
        px.off()
    time.sleep(0.05)
```

16. What is the core problem with using `time.sleep()` when you want two things to happen at the same time? How does `AsyncRunner` solve it?

ANSWER

`time.sleep()` freezes the entire program — nothing else can run during the pause.

`AsyncRunner` uses cooperative multitasking: each async task runs until it hits `await`, then pauses and lets other tasks run. No task is ever blocked waiting for another to finish sleeping.

17. What are the two changes you must make to a regular function to make it work with `AsyncRunner`?

ANSWER

1. Change `def` to `async def`.
2. Replace `time.sleep()` with `await AsyncRunner.sleep()`.

18. (Stretch) Explain in your own words what `await` means in the context of `AsyncRunner`. Use the 'worker' analogy from the Primer if it helps.

ANSWER

When a task hits `await`, it signals to the runner that it is waiting and another task can run — like a worker raising their hand to say 'I'm done for now, someone else go.' The runner cycles through all tasks, switching only at `await` points. No task ever interrupts another mid-instruction.